Exercise 3

Arrays, Life, and Tetris

Released: 4 November 2026 | Due: 18 November 2026, 23:59

Learning objectives

Use functions and two-dimensional arrays to represent images, simulate a cellular automaton, and build an interactive game on an 8×8 LED matrix.

General instructions

The teaching assistant responsible for this exercise is Itay Shedlezki.

The exercise has two parts: Game of Life and Tetris. Implement both in `ex3.c`.

Clone the public course repository at github.com/intro2cs-biu/ex3 and work on your local copy:

```
git clone https://github.com/intro2cs-biu/ex3.git
cd ex3
```

You cannot push changes to the course repository. If you copy the repository to your own GitHub account, that copy must be private. Making solutions publicly accessible is strictly prohibited because other students may copy them, which can lead to an academic-integrity review for everyone involved.

Please post questions about the exercise **publicly** in the course forum.

Read the Coding Style file on the course site and work accordingly. Code that does not follow it may receive a point reduction. Compile with `-Wall -Wextra -Werror` and resolve every warning and error. Testing on the departmental server is highly recommended. An exercise containing warnings or errors may receive a penalty of up to 100 points.

Submit by **18 November 2026 at 23:59** to **both** the course submission system (one ZIP containing `ex3.c`) and the Google Form (`ex3.c` uploaded directly, not inside a ZIP). See the Submission section for full instructions.

Late submissions are penalized as follows:

- One day late: 10 points. The maximum grade is 90.
- Two days late: 20 points.
- Three days late: 30 points.
- **You cannot submit this or any other exercise more than three days late.**

Requests for an extension, for justified reasons only, must be sent to intro2cs.biu@gmail.com and include:

- full name,
- submission-system username, and
- ID number

Important

The purpose of this introductory course is not only to learn C syntax. More importantly, it is to develop **logical thinking, algorithmic thinking, problem-solving skills, and the ability to debug your own programs**. Getting stuck, reading an error, reconsidering an algorithm, asking a question, and trying again are essential parts of that learning process. AI code generation is especially tempting when you are stuck, but allowing it to finish a function, generate an algorithm, or fix a bug removes precisely the work through which these skills develop. Therefore, do not use generative-AI systems to produce, complete, rewrite, or debug your exercise, even if you intend to read or modify the generated result afterward. Your submitted work must be written by you. Do not share code with other students. You are reminded that thinking together is a blessing. Working together is cheating. An exercise identified as copied will receive **0** for everyone involved, and additional action may be taken.

Required language

Use only material covered in lectures and recitations (tirgulim) up to and including 4 November 2026, including functions, conditions, loops, and fixed-size arrays. Do not use variable-length arrays.

The only standard C headers allowed are `stdio.h` and `stdlib.h`. Use the supplied course headers for the matrix, joystick, Arduino, and serial functions. In particular, do not include `stdint.h`, `ctype.h`, `limits.h`, or `string.h`. Use `scanf` for numeric and character input. For the hexadecimal image, use the course function described in Part 1.

Organize your code into functions that encapsulate distinct pieces of logic. When the same logic serves multiple purposes, reuse a function rather than duplicating its code.

Files, compilation, and serial communication

Download the files for the exercise from the Ex3 GitHub repository. It contains:

```
ex3/
|-- ex3.c (empty starter)
|-- executables/
|   |-- Arduino
|   |-- macOS (Apple Silicon)
|   |-- macOS (Intel)
|   |-- Linux
|   `-- Windows
`-- tests/
    |-- inputs/
    `-- expected/
```

The executables let you inspect the reference behavior on your platform. The tests folder contains sample inputs and matching expected outputs.

Keep the course library folder `intro2cs-lib` next to the `ex3` folder. Use the library package for your operating system. From the `ex3` directory, compile, create the HEX file, find the board's port, upload, and open the serial monitor:

```
avr-gcc -mmcu=atmega328p -DF_CPU=16000000UL -Os -std=c99 -Wall -Wextra -Werror -I../intro2cs-lib/headers ex3.c ../intro2cs-lib/intro2cs-arduino-lib.a -o ex3.elf
avr-objcopy -O ihex -R .eeprom ex3.elf ex3.hex
arduino-cli board list
avrdude -p atmega328p -c arduino -P PORT -b 115200 -D -U flash:w:ex3.hex:i
arduino-cli monitor -p PORT --config baudrate=9600
```

Replace `PORT` with the port reported by `arduino-cli board list`, such as `COM3`, `/dev/ttyACM0`, or `/dev/cu.usbmodem...`. Close the terminal monitor before uploading. After uploading, run the monitor command above with the correct port. Opening the monitor may reset the board.

Program interaction and pixel representation

Print the main menu exactly as follows, with one blank line before it:

```
1. Game of Life
2. Tetris
3. Exit
```

For choice 3, print `Goodbye!` followed by a newline.

When a task finishes, print the following prompt and wait for the user to press Enter. Then clear the matrix and return to the main menu:

```
Press Enter to continue.
```

Choosing 3 from the main menu clears the display and terminates the program.

Assume all input is valid and complete: menu choices are listed options, numbers are in the specified ranges, and static piece sequences contain 1 to 100 uppercase piece letters followed by a period. You do not need to validate input, handle invalid input or end-of-file, print error messages, or implement input-recovery loops. Generation counts and delays are nonnegative integers representable by `int` on the Arduino. Enter each numeric input on its own line, with no trailing spaces. End the static sequence line immediately after its period. After a task, press Enter on an empty line to continue. Every message shown in this handout ends with a newline unless stated otherwise.

Matrix setup and image input

Connect the matrix and joystick as shown below. The joystick is used in Part 2.

Initialize the matrix with `matrixBegin(NULL, NULL)`. Rows and columns are numbered 0 through 7, with row 0 at the top and column 0 at the left.

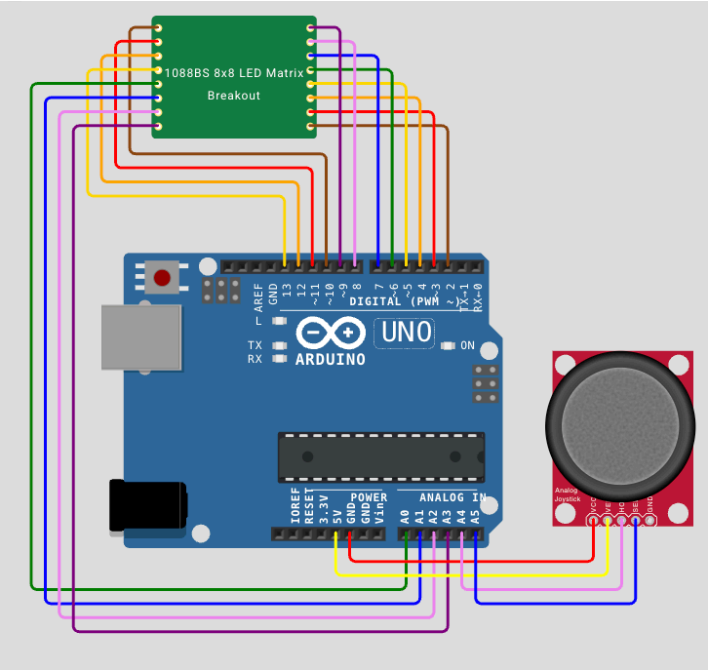


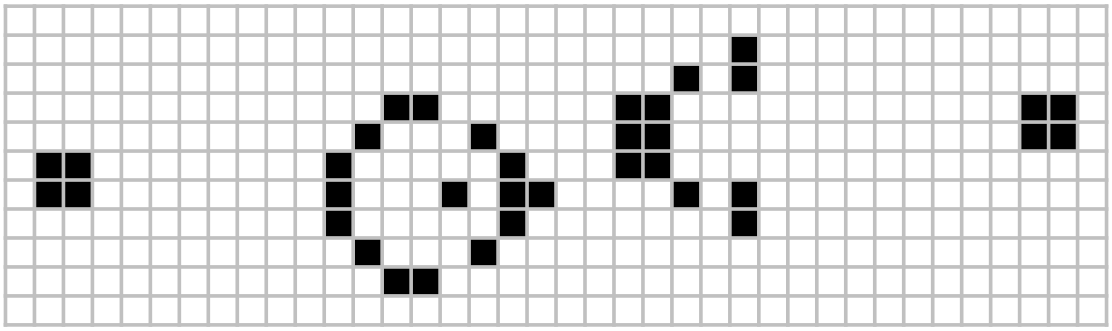
Figure 1: Required wiring for the LED matrix and joystick.

The library provides `matrixSetFromPixelArray` for displaying an 8×8 array of `unsigned char`, with 0 for an unlit pixel and 1 for a lit pixel. Use only this function to update the entire matrix at once.

Part 1 – Game of Life

John Conway's Game of Life is a cellular automaton introduced to a wide audience by Martin Gardner in 1970.[1] It is a simulation rather than a game with moves: an initial pattern evolves on its own according to a few local rules. Some patterns disappear, some settle, and others oscillate or travel. Here, each LED is a cell, letting you watch how simple rules create changing patterns.

Optional extra reading: Conway's Game of Life on Wikipedia includes animations of oscillators, spaceships, and glider guns.



A Gosper glider gun: a repeating pattern that emits moving gliders.
This larger-board illustration is for inspiration; implement the 8×8 board below.

Print the following prompt and read a 64-bits (16 digits) hexadecimal image, with the top row first:

```
Enter the image in hexadecimal (top row first):
```

Read it with `serialReadUnsignedLongLong(&image, 16)` from `serial.h`. Leading zeros may be omitted. After reading the initial image, print these prompts and read a decimal integer after each:

```
Enter the number of generations:
Enter the time per generation in milliseconds:
```

Display the initial image. Before each requested generation, wait the given number of milliseconds, then display the next generation. If at least one generation was requested, wait the given interval once more after displaying the final generation. Then print the continue prompt, wait for Enter, clear the matrix, and return to the main menu. Zero generations means that only the initial image is displayed, with no generation delay.

Each matrix position is a cell with eight neighbors, including diagonals. The board **wraps around both horizontally and vertically**: the left and right edges are adjacent, and the top and bottom edges are adjacent. This also applies diagonally. For example, the eight neighbors of cell `(0, 0)` are `(7, 7)`, `(7, 0)`, `(7, 1)`, `(0, 7)`, `(0, 1)`, `(1, 7)`, `(1, 0)`, and `(1, 1)` (row, column). These wrapping rules apply only to Game of Life; Tetris pieces must stay within the board. For example, three live cells at `(0, 7)`, `(0, 0)`, and `(0, 1)` form a horizontal blinker across the left/right edge. Its next generation is `(7, 0)`, `(0, 0)`, and `(1, 0)`, crossing the top/bottom edge. The following generation restores the initial pattern.

The next generation obeys these rules:

- A live cell survives if it has exactly two or three live neighbors.
- A dead cell becomes live if it has exactly three live neighbors.
- Every other cell is dead in the next generation.

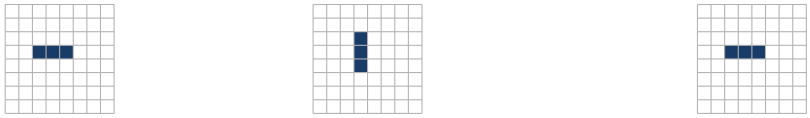
[1] Game of Life, Scholarpedia.

All cells change simultaneously: every result is determined from the same previous generation.

Example

A blinker: a pattern that repeats every two generations. Enter `0000003800000000` as the image, 2 generations, and a 500 ms interval. Filled cells are alive; all other cells are dead. Each grid below is the full 8×8 board, with no gaps between cells.

Initial: 0 ms **Generation 1: 500 ms** **Generation 2: 1000 ms**



In the first step, the middle live cell survives because it has two live neighbors. The two end cells die because each has only one. The dead cells immediately above and below the middle become alive because each has three live neighbors. All these changes use the initial board, simultaneously. The next generation restores the horizontal pattern.
After displaying generation 2, wait another 500 ms. At 1500 ms, print `Press Enter to continue.` and wait for Enter before clearing the board.

Part 2 – Tetris



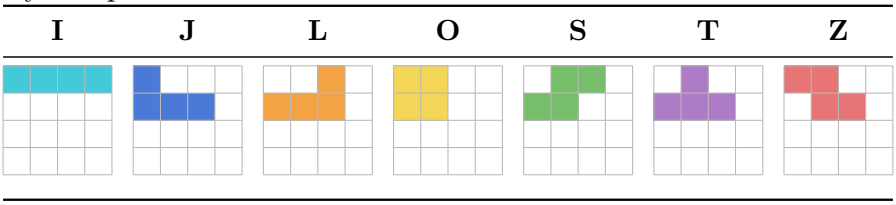
Since its 1984 debut, Tetris has become one of the world's most popular and influential video games. Its simple rules, escalating challenge, and success across computers, consoles, and handheld devices made it a landmark in the history of games.[2]

In Tetris, the player moves and rotates falling pieces to complete horizontal rows. Completed rows disappear, leaving room for more pieces. The game ends when the next piece cannot enter the board. In this part, implement an 8×8 version controlled by the joystick.

The board and the pieces

The board contains **fixed cells** from pieces that have already landed. One **active piece** moves above them. Display both together, but add the active piece to the fixed board only when it locks, as explained below.

There are seven piece types, each containing four occupied cells. The grids below show their initial orientations in a 4×4 array. White cells are empty padding; colors are only for this handout. On the matrix, every occupied cell is lit.



The position `(x, y)` is the board column and row of the piece array's top-left corner. Row 0 is at the top; column 0 is at the left. Every new piece starts at `(2, 0)` in its initial orientation.

[2] About Tetris; The History of Tetris.

A position is legal if every occupied piece cell is inside the board and does not overlap a fixed cell. Empty padding may extend outside the board. Apply this same rule to spawning, movement, and rotation.

Starting a game: choose the difficulty

Start with an empty board and a score of zero. Print the following menu and read the user's choice:

```
Choose difficulty:
1. Easy (250 ms)
2. Normal (150 ms)
3. Hard (75 ms)
4. Adaptive (250 ms initially)
```

Choose the piece order

After reading the difficulty, print:

```
Choose piece order:
1. Random
2. Static
```

Random order. If the user selects Random, print:

```
Enter seed (-1 for current time):
```

Read the seed and call `srand` **only once per game, before generating the first piece**. Do not call it again for each new piece. For seed `-1`, use `(unsigned int)millis()` as the seed; otherwise use the entered seed converted to `unsigned int`. For each new piece, use `rand` to select an ID from 0 through 6, with this mapping:

ID	0	1	2	3	4	5	6
Piece	I	J	L	O	S	T	Z

Repeated pieces are allowed. The same seed repeats a sequence on the same platform. Sequences may differ across platforms.

Static order. If the user selects Static, print:

```
Enter up to 100 piece letters (I, J, L, O, S, T, Z), ending with .:
```

Read a sequence of 1 to 100 piece letters followed by a period. Supply pieces in the entered order. After supplying the last piece in the sequence, repeat the same order from the beginning. For example, `IOT.` produces I, O, T, I, O, T, and so on. You may assume the letters are uppercase. Whitespace between letters may be ignored by reading each letter with `scanf(" %c", &letter)`.

Assume all input is legal, including the difficulty, piece-order choice, seed, and static sequence. No input validation is required.

After successfully choosing the piece order, print:

```
Tetris game starts: left/right to move, up to rotate (up-left counterclockwise), down to
    fall faster.
```

Then obtain the first piece and try to place it at its starting position.

Joystick controls and rotation

Use the wiring shown earlier: X on A4 and Y on A5. The joystick button is unused.

Direction	Action
Left / right	Try to move one column each update, only while up is inactive.
Up-left	Try to rotate counterclockwise; no horizontal movement.
Up-right	Try to rotate clockwise; no horizontal movement.
Up alone	Try to rotate clockwise; no horizontal movement.
Down	Move down one row each update while airborne.

While up is active, do not move horizontally. Up-left requests a counterclockwise rotation; up alone or up-right requests a clockwise rotation.

Rotate by 90° in the selected direction, then remove empty rows above and empty columns to the left of the rotated shape. This keeps the shape aligned to the top-left of its array and preserves its board position `(x, y)`, rather than displacing it because of empty padding introduced by rotation. For example, rotating the initial horizontal I produces a vertical I occupying the first column of its piece array, starting in its top row. The O piece keeps its appearance and position.

Accept the rotation only if the resulting position is legal. Otherwise keep the previous shape. Do not shift the piece sideways or upward to make a rotation fit.

Falling and landing

A new piece appears at `(2, 0)` in its initial orientation. It remains active while the player moves and rotates it, falls toward the bottom of the board, and eventually locks in place. The game advances in *updates*: each update reads the joystick, applies the input, handles falling or landing, and ends with a delay.

Normal falling. Each new piece begins a falling phase of seven updates. During the first six, the player can move or rotate the piece while it stays at the same height. On the seventh, apply the input and then move the piece down one row, provided it can fall. Begin a fresh seven-update phase after every downward move.

Faster falling. If down is active while the piece can fall, move it down one row in that update, without waiting for the seventh update. This also starts a fresh falling phase. Holding down therefore moves the piece down one row per update.

Landing. A piece is *grounded* when it cannot move down another row. After it lands, allow four more updates before locking it to the board. In each update, apply movement or rotation first, then check support. If the piece remains grounded at all four checks, lock it after the fourth check. The update that moves the piece down onto support does not count toward these four updates. Movement or rotation can also make an airborne piece grounded. Since support is checked after applying that input, this same update counts as its first grounded check. Holding down does not shorten the landing allowance. This allowance also applies when the piece would complete a row: rows are cleared only after the piece locks to the board.

Example

Suppose the piece moves down onto the floor in update 10:

Update	What happens
10	The piece is airborne when support is checked, then falls onto the floor. This update does not count toward the landing allowance.
11	Apply input. If still grounded, this is grounded update 1.
12	Apply input. If still grounded, this is grounded update 2.
13	Apply input. If still grounded, this is grounded update 3.
14	Apply input. If still grounded, this is grounded update 4: lock the piece to the board.

The same sequence applies when landing on fixed cells instead of the floor.

Moving off support. Moving or rotating while still grounded does not restart the four-update landing allowance. If the piece becomes airborne (able to move down another row), that allowance resets. On its next landing, it receives a fresh four-update allowance.

The seven-update falling phase is separate from the landing allowance. **Save the current falling phase's progress while grounded:** this progress stays unchanged during grounded updates; do not advance or reset it. If movement or rotation makes the piece airborne, that same update advances the falling phase again. Only a downward move resets the falling phase; a new piece also starts with zero completed updates.

For example, suppose a piece has completed four airborne updates since its last downward move, then moves sideways onto support. Its falling progress stays at four during grounded updates. If it moves off support before locking, that update is its fifth airborne update. Without down input, it falls on its seventh airborne update, then starts a fresh phase.

If instead the piece became grounded by falling onto support, that downward move already reset its falling progress to zero. Moving off that support therefore begins with airborne update 1, not with progress from before the landing.

Difficulty sets the speed. The flow above is the same at every difficulty: seven updates per falling phase and four updates after landing. The selected difficulty changes only the delay at the end of each update:

Difficulty	Delay per update
Easy	250 ms throughout the game
Normal	150 ms throughout the game
Hard	75 ms throughout the game
Adaptive	Starts at 250 ms and decreases as rows are cleared

For example, at Normal difficulty, normal falling takes $7 \times 150 = 1050$ ms per row; holding down reduces this to 150 ms per row.

For Adaptive difficulty, divide the delay by 1.2 after each cleared row, round down to whole milliseconds, and keep it at least 50 ms. Apply this change once per row, even when several rows are cleared together. For example, the first two cleared rows reduce 250 ms to 208 ms, then to 173 ms. Finish the locking update using its existing delay; the reduced delay takes effect with the next piece. Each new game starts with its selected difficulty's initial delay.

One game update

For each active piece, repeat the following steps until it locks:

1. **Display and read input.** Display the fixed board with the active piece. Call each direction function **exactly once per iteration**, in this order:

`joystickLeft()`, `joystickRight()`, `joystickUp()`, `joystickDown()`.
Use those results throughout the update. The simulator consumes input on each call, so additional calls would make the output inconsistent with the supplied tests.

2. **Apply movement or rotation.** If up is inactive, try horizontal movement. If up has just become active, try the selected rotation instead. Accept only legal positions. **Holding the joystick up causes only one rotation, not continuous rotation**, even when a new piece appears. To rotate again, move the joystick out of the up direction, then move it up again. This also applies to up-left and up-right.

3. **Apply falling or landing.** After movement or rotation, check whether the piece is grounded. If it is already grounded, count **this update** toward the four-update landing allowance and lock it on the fourth consecutive grounded check. Otherwise, apply the falling phase described above. If that downward move lands the piece on support, its four-update allowance begins in the **next update**.

4. **Wait.** Call `delay` with the current difficulty delay, including in the update that locks the piece to the board.

Do not use `millis()` to measure elapsed time for falling or landing; use only update counts instead.

Locking, clearing rows, and continuing

When a piece locks:

1. Add its occupied cells to the fixed board.
2. Remove every full row (a row with eight occupied cells). Each removed row adds one point to the score. Rows above a removed row move down, and an empty row enters at the top. Rows below it keep their positions.
3. Display the resulting fixed board once, without an active piece, then complete the update's current difficulty delay. This is the delay from the update loop, not an additional delay.
4. Obtain the next piece in its initial orientation at `(2, 0)`. If it fits, begin its first update with a fresh falling phase and landing allowance. For Adaptive difficulty, apply any delay reduction from the cleared rows before starting this piece. The display call at the start of that update shows the board together with the new piece; do not add a separate display call when obtaining the piece. Otherwise, end the game.

Game over

If a new piece does not fit at its starting position, do not place it. Leave the final fixed board displayed without another display call, and print:

```
Game over.
Rows cleared: N
```

Replace `N` with the total number of cleared rows in decimal. Then print the usual continue prompt, wait for Enter, clear the matrix, and return to the main menu. There is no joystick gesture or button for leaving a game early.

Where do I begin?



Not sure where to begin? There are many details here, but you do not need to solve them all at once. Start by organizing the specifications into a main flow in your own words: how does a game start, what repeats, and when does it end? Think about which loops you need and which conditions decide what happens next.

Then look for smaller blocks of logic. You may find it helpful to have functions for rotating a piece, checking collisions, or clearing a full row. Choose one small task, implement it, and check it before moving on. A working game grows from these small steps.

Verify behavior with the simulator

You can use the course simulator to verify your program on your computer. With the library folders arranged as described earlier, compile and run from `ex3`:

```
gcc -std=c99 -Wall -Wextra -Werror -I../intro2cs-lib/headers ex3.c ../intro2cs-lib/intro2cs-arduino-simulator-lib.a -o ex3
./ex3
```

You can also run the matching reference executable from the repository's `executables/` folder. Use the sample input and expected output files in `tests/` to check your implementation.

For Tetris, the simulator consumes one character for each direction check, so supply four characters per update in left/right/up/down order. An `a` in the first position activates left, `d` in the second activates right, `w` in the third activates up, and `s` in the fourth activates down. Use `.` for an inactive direction. Two active characters in one update represent a diagonal:

```
....    no direction
a...    left
.d..    right
..w.    up only; rotate clockwise
a.w.    up-left; rotate counterclockwise only
.dw.    up-right; rotate clockwise only
...s    faster falling
```

Whitespace is ignored by these direction checks. To rotate again, include an update with up inactive between rotations. The simulator reports delays without waiting in real time. A nonnegative seed repeats a sequence on the same platform. Sequences need not match across platforms.

Check corners and edges in Life, rejected moves and rotations, bottom-row landing, rotation during the lock delay, consecutive full rows, and a blocked spawn. The automatic submission check does not replace these checks or determine the complete grade.

Submission

You may develop your code in any environment. You must know how to copy your C source file to the departmental server, compile it there with the course hardware library, and submit it to both

systems below. Testing on the server is highly recommended to ensure there are no warnings or errors.

Submit by **18 November 2026 at 23:59**. Submit only the file containing your code, not files created by an editor, IDE, compiler, operating system, or simulator. **DO NOT** use Hebrew characters in file or folder names. A file whose path contains Hebrew characters cannot be copied reliably to the university servers.

Your submission must contain exactly one source file, with its original name:

- `ex3.c`

You must submit your solution to both systems:

1. Course submission system: upload one ZIP archive whose top level contains exactly `ex3.c`.
2. Google Form: upload `ex3.c` directly, **not inside a ZIP archive**.

Submit the same version of `ex3.c` to both systems. You may submit multiple times; only your last submission in each system will be considered. If you update your solution, resubmit the updated file to both systems so that your final submissions match.

Do not include the library directory, simulator archive, executables, object files, ELF files, HEX files, editor settings, or enclosing folders. Open the ZIP after creating it and confirm its contents. Your source must compile without warnings on the departmental Linux server.

☐ The ZIP contains exactly `ex3.c`.

☐ The program compiles with `-Wall -Wextra -Werror`.

☐ Both modes were tested on the Arduino with the matrix and joystick.

☐ Any personal GitHub repository containing my work is private.

☐ The same final file was submitted to both systems: inside a ZIP in the course submission system and directly in the Google Form.